

Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

This tutorial provides a basic introduction to representing classes and relationships using the Unified Modeling Language (UML). You will see this notation used in subsequent course lectures. UML is the standard industry diagramming notation for object-oriented software development projects.

In object-oriented programming (OOP) we begin solving problems by identifying and modeling relevant objects as well as the attributes and behaviors of those objects. We may also determine how different objects are related to one another. This is called object-oriented design (OOD), and it's a very important and necessary step that precedes the programming step. Actually, I've used the word "objects" here, even though what we design are "classes", but "class-oriented" design and "class-oriented" programming don't sound nearly as sexy as "object-oriented".

In OOD, designs are represented using UML *class diagrams*. Class diagrams illustrate very concisely the attributes and methods a class will have. They can also illustrate the relationships between classes used in the solution of a problem.

Let's take an example to see how we can perform OOD to model a cat. We begin the design process by deciding that we need to design a cat. Then, we identify the relevant attributes and behavior needed to model a cat. For discussion purposes we'll keep it simple. Let's use the attributes below to model a cat.

Cat Attributes

Breed
Age
Weight
Sex
Name

And, let's assume we're only interested in two behaviors for our example:

Cat Behaviors

Eating
Meowing

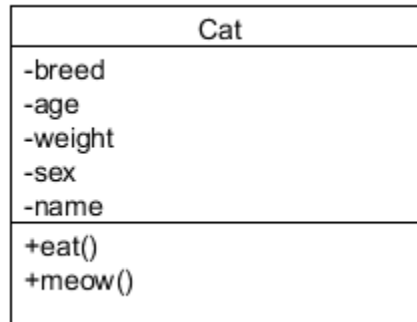
To document our design we will use a UML class diagram, which will look like this for the cat example:

Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

A Simple Class Diagram



The class diagram is very simple, but it documents our cat class design concisely. The diagram uses a rectangle as an icon for our model of a cat. The rectangle contains three compartments. The top compartment contains the name of the thing being modeled...in our case a cat. The next compartment contains a list of the attributes we are including in our model. And the bottom compartment contains a list of relevant behaviors. Note that parentheses were added to the behavior names to indicate that they will correspond to methods when we actually write the code.

I'm sure you noticed the use of the hyphens (-) and plus signs (+) in the diagram. These are used to indicate a class member's visibility or access mode. A plus sign (+) indicates *public* visibility, a hyphen (-) indicates *private* visibility, and a pound sign (#) indicates *protected* visibility. The meaning and mechanics of public, private, and protected are covered in lectures and in the textbook.

Now, let's refine the above design a bit. Before turning the design into code, we need to determine the types of each attribute, and we need to determine if the methods will take parameters and return anything. Let's start with the attribute types. The attribute types can be primitive or non-primitive types. A good type for the age attribute would be an integer, since we commonly think of ages as being described in integer years. For the weight attribute, let's

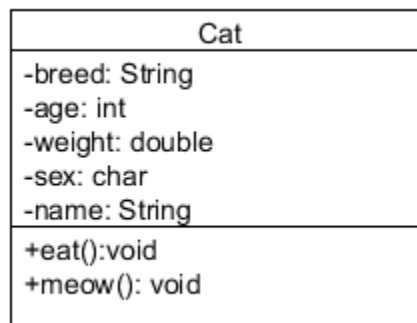
Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

use a double so that we can keep track of a cat's weight more accurately...a whole pound of weight can be a lot of weight difference for a cat. For the sex attribute, let's use a character type...'f' for female and 'm' for male. And for the breed and name attributes, let's use a String type. Regarding the methods, let's assume that the eat method will display output like "Munch, munch" and that the meow method will display "Meow, meow", and that we won't pass anything into the methods. So, we'll specify that these methods have a void return type. Our refined model is as follows:

A Refined Class Diagram



This refined diagram tells us exactly what types to use for each attribute, and what parameter and return types will be used when we ultimately write the code for our Cat class.

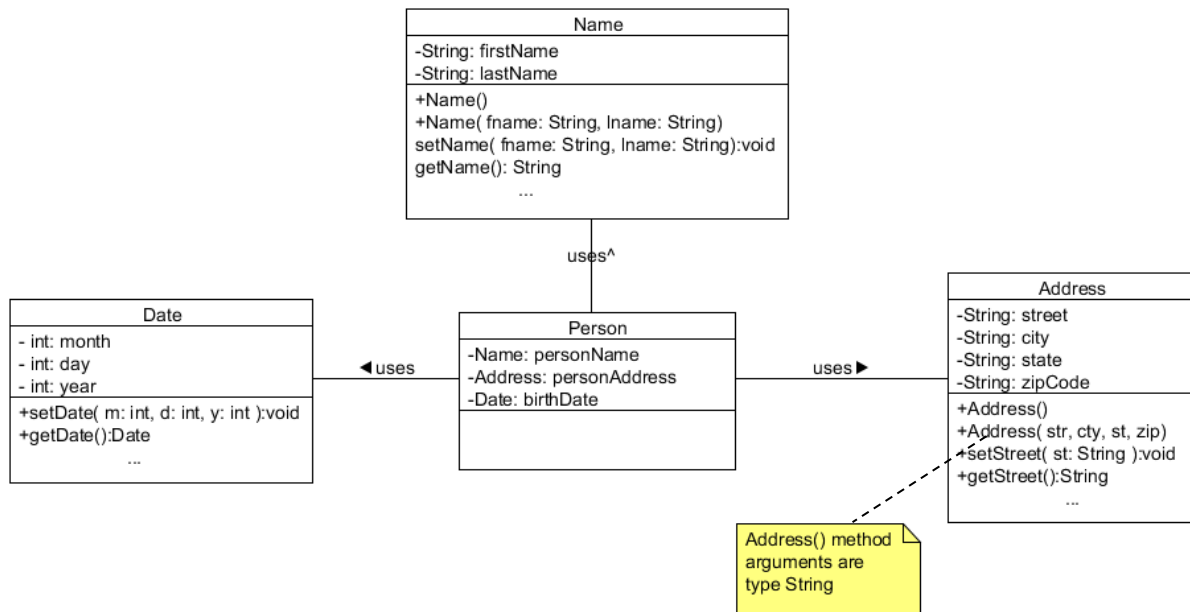
Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

Illustrating Containment

In an earlier lecture, you learned about containment and aggregation relationships. A class diagram can also be used to show that a design is based on containment or aggregation, but only containment will be illustrated here. Suppose we had a Name class, Address class, and Date class...and we wanted to reuse those classes to model a Person class. The following class diagram can be used:



When containment is used in a design, we sometimes say a “uses” (or a “has a”) relationship exists. In this example, the Person class “uses” a Date object, a Name object, and an Address object. (No methods are shown for the Person class for brevity and because the purpose of this diagram is to illustrate the concept of containment.)

Note in the class diagram that Java types are specified for the class attributes and some of the methods. As an example, the `setDate()` method in the Date class indicates that the method takes three integer arguments: the first argument corresponding to the month, the second corresponding to the day, and the third corresponding to the year. The return type of this method is specified as *void*, so the method does not return a value.

In the Name class, the first two methods are constructors. It is easy to know this because the method names are the same as the class name. Also, by convention, constructors are generally shown as the

Introduction to Programming Using Java

605.201

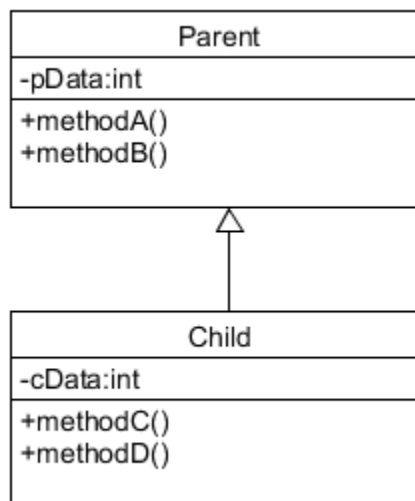
Unified Modeling Language (UML) Basics

first methods for a class. Note that the constructors do not have a return type since constructors can't return a value.

Note the second constructor for the Address class. Only the method parameter names are indicated in the diagram...but not the types of the parameters. The UML syntax that we have been using is very flexible regarding how much detail needs to be shown. The more detail that is shown the easier the transition from design to code in general, since we have to make fewer programming decisions. In this example, the parameter types are not shown for two reasons: first, for the sake of brevity; and second, to introduce how comments can be added to a class diagram. In this case, the comment provides information about the parameter types.

Illustrating Inheritance

Another type of relationship that can be illustrated on a UML class diagram is inheritance. Technically, this is called generalization/specialization in the UML because a superclass (or parent class) is supposed to be more “general” than a subclass (or child class), and a subclass is supposed to add attributes and methods that make it more “specialized” than its parent. Inheritance is illustrated in the diagram below.



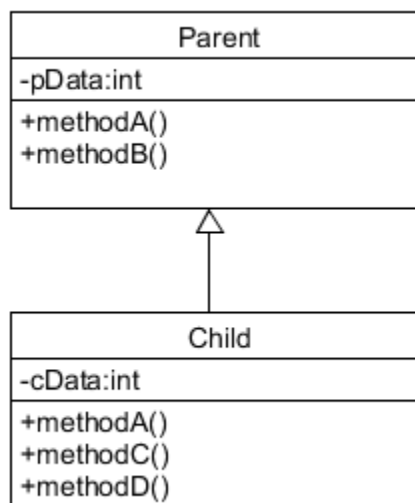
Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

This diagram illustrates that the Parent class has a private data attribute and two public methods. These class members are inherited by the Child class and it defines its own data member and two more methods. Note that the inherited data member is private. This means that the Child class can't access the data member by name, but can only have access to it if methodA or methodB provide access. By the way, when a parent class has private data members, some textbook authors say that those members are not inherited. This is not quite accurate, in my opinion, since storage is reserved for such members when a child class is instantiated...but I just wanted to make you aware of it. Note also in this diagram that none of the Parent class members are shown as Child class members. Because inheritance is the relationship being used here, it is assumed that the Parent class members become members of the Child class. This is important.

There is also an important convention that we use in practice to indicate that a child class overrides a method inherited from a parent. To indicate a method is overridden, the method signature is repeated in the Child class, as illustrated below.



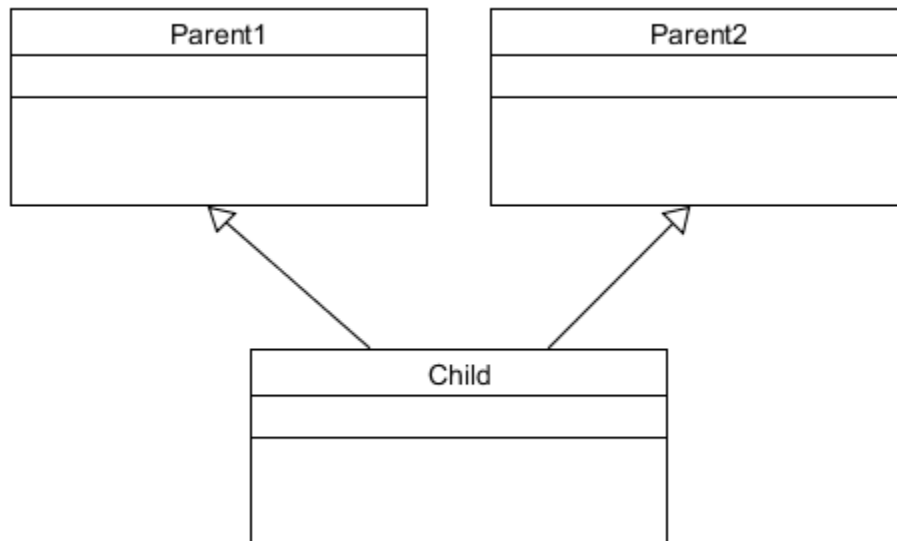
Introduction to Programming Using Java

605.201

Unified Modeling Language (UML) Basics

The above diagram repeats the signature for method in the Child class. This indicates that the Child class will override the inherited method of the same name and provide its own implementation.

The diagram below indicates what is called multiple inheritance. In a multiple inheritance relationship, a child class inherits from more than one direct parent. This diagram shows the Child class inheriting from two direct parent classes...Parent1 and Parent2.



Now, Java does not implement multiple inheritance for ordinary classes, but other object-oriented languages, like C++, do.

This brief tutorial is intended to provide a quick introduction to using UML to represent classes and relationships. There's a lot more to UML, and a bit more will be introduced as the course progresses. It's important to understand the UML basics because UML is commonly used in practice, it is a very concise way to perform and illustrate OOD, and it will be used to introduce additional OOD and OOP concepts in future course modules.